Table of Contents

hw4_me501_Q5.m 1
Setup 1
Calculate the discretized non-autonomous system via the zero order hold 1
Calculate a sequence of control inputs 2
Simulate 2
Plot 3
Sanity check 3

hw4_me501_Q5.m

(C) Andrew Sabelhaus, 2026

```
clear all;
close all;
clc;

disp('Feedforward control of a linearized system.')
```

```
Feedforward control of a linearized system.
```

Setup

```
% Continuous-time A and B matrices:
A = [0 1;
     4 -3];
B = [0; 1];

% We need to specify a sampling time:
dt = 0.01;

% We will also need an initial state:
x0 = [pi/4; 0];

% specify a total time of the simulation:
tmax = 10;
n = tmax/dt; % number of timesteps, for loop-ing
% somehow matlab doesn't round correctly. sometimes...
n = round(n);
```

Calculate the discretized non-autonomous system via the zero order hold

```
% Use the trick from the problem set to calculate both Ad and Bd at once,
% using the matrix exponential.
M = dt*[A, B; zeros(1,3)];
eM = expm(M);
```

```
% Extract the top-left 2x2 matrix as Ad and the top-right 2x1 vector as Bd
Ad = eM(1:2,1:2);
Bd = eM(1:2,3);
```

Calculate a sequence of control inputs

```
% The number of timesteps here is T=n.
xf = [0;0];
u_Tk = soln_u_min_sequence(Ad, Bd, n, x0, xf);
% remember, the sequence of inputs is opposite in time!
% we need to apply u_kT rather than Tk.
u_kT = flip(u_Tk);
```

Simulate

```
% Here, we compare the discretized dynamics versus the original nonlinear
% dynamics. Do our linearized predictions hold true?

% insert our initial condition as the 1st element in traj
% note that traj will have n+1 columns.
x_trajlin = zeros(size(x0,1), n+1);
x_trajlin(:,1) = x0;

% and a trajectory of the system with the forward euler integrated
% nonlinear dynamics
x_trajnonlin = zeros(size(x0,1), n+1);
x_trajnonlin(:,1) = x0;

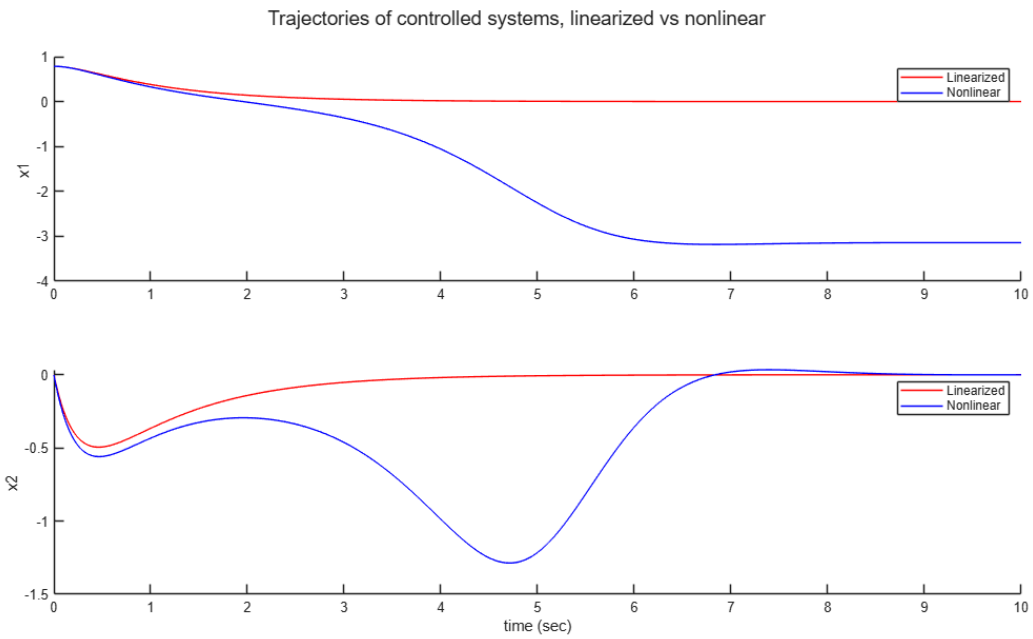
% Simulation (rollout) of our exact discretization of the linearized
% dynamics
for k=1:n
    %%%%% DYNAMICS
    x_kp1 = Ad*x_trajlin(:,k) + Bd*u_kT(k);
    % save the result
    x_trajlin(:,k+1) = x_kp1;
end

% Forward euler, which works as a baseline integration method always
for t=1:n
    %%%%% DYNAMICS
    % for more clarity:
    xt = x_trajnonlin(:,t);
    x1 = xt(1);
    x2 = xt(2);
    % The zero order hold means we simply apply the corresponding u as we
    % would in discrete time at this value of t=k*dt.
    bolddotx_t = [x2; 4*sin(x1) - 3*x2 + u_kT(t)];
    % Forward Euler
    x_trajnonlin(:,t+1) = x_trajnonlin(:,t) + dt*bolddotx_t;
end
```

Plot

```
% We want to see all three states
figure;
hold on;
sgtitle('Trajectories of controlled systems, linearized vs nonlinear')
t = [0:dt:tmax];

% number of states here will be N
N = size(x0,1);
for i=1:N
    subplot(N,1,i)
    hold on;
    plot(t, x_trajlin(i,:), 'r');
    plot(t, x_trajnonlin(i,:), 'b');
    % a neat trick to obtain the string "x1", "x2", etc
    ylabel(strcat("x", num2str(i)));
    legend("Linearized", "Nonlinear")
end
xlabel('time (sec)')
```



Sanity check

```
% Do either of the trajectories end at the origin?
disp("Both trajectories should end at 0. Do they?")
format long g
disp('Linearized dynamics (prediction):')
x_trajlin(:,end)
disp('Nonlinear dynamics (true system response):')
x_trajnonlin(:,end)
```

```
Both trajectories should end at 0. Do they?
Linearized dynamics (prediction):

ans =

     -2.40184051727664e-05
     -2.40184032459465e-05

Nonlinear dynamics (true system response):

ans =

         -3.14123304110348
     -0.000625459643550622
```

Published with MATLAB® R2026a